

PYTHON ESSENTIALS FOR ODOO DEVELOPMENT

Python for Odoo Developers

Core Python concepts used every day in Odoo

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026



Agenda

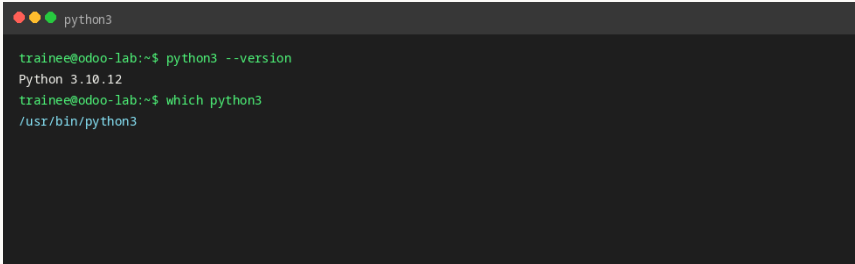
- 1 Section 1: Running Python programs — 4 topics
- 2 Section 2: Basic values and text — 10 topics
- 3 Section 3: Collections of data — 9 topics
- 4 Section 4: Control flow — 8 topics
- 5 Section 5: Functions and reusable code — 7 topics
- 6 Section 6: Objects, errors, modules, and Odoo habits — 11 topics

1 Section 1

Running Python programs

1.1 What Python and the interpreter are

- Python is a programming language used to write instructions for a computer.
- The interpreter is the program that reads and runs Python code.

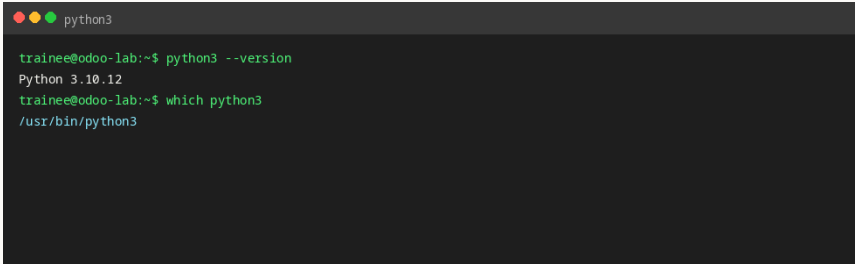
A terminal window with a dark background and light green text. The window title bar shows three colored circles (red, yellow, green) and the text 'python3'. The terminal content shows two commands and their outputs: 'python3 --version' returns 'Python 3.10.12', and 'which python3' returns '/usr/bin/python3'.

```
python3

trainee@odoo-lab:~$ python3 --version
Python 3.10.12
trainee@odoo-lab:~$ which python3
/usr/bin/python3
```

1.2 Run python and python3

- On many Linux systems, python3 is the command for modern Python.
- The python command may be missing or may point to a different version.

A terminal window with a dark background and light green text. The window title bar shows three colored circles (red, yellow, green) followed by the text 'python3'. The terminal content shows a user named 'trainee' at a prompt 'trainee@odoo-lab:~\$' running the command 'python3 --version', which outputs 'Python 3.10.12'. The user then runs 'which python3', which outputs '/usr/bin/python3'.

```
python3

trainee@odoo-lab:~$ python3 --version
Python 3.10.12
trainee@odoo-lab:~$ which python3
/usr/bin/python3
```

1.3 First program

- A Python program is a text file containing Python statements.
- The print function displays output in the terminal.

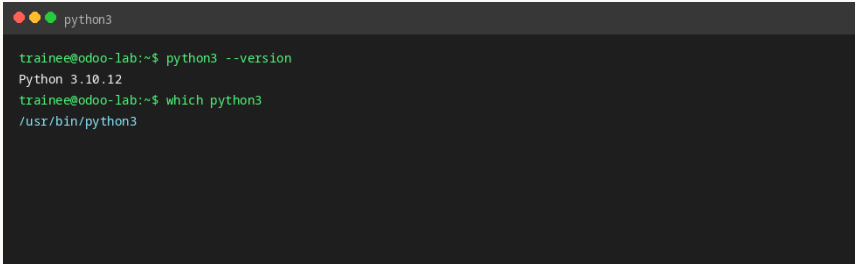
A terminal window with a dark background and light green text. The window title is 'python3 hello.py'. The prompt is 'trainee@odoo-lab:~\$'. The user enters 'cat hello.py' and the output is 'print('Hello, Merit Advisory')'. The user then enters 'python3 hello.py' and the output is 'Hello, Merit Advisory'.

```
python3 hello.py

trainee@odoo-lab:~$ cat hello.py
print('Hello, Merit Advisory')
trainee@odoo-lab:~$ python3 hello.py
Hello, Merit Advisory
```

1.4 Python 2 vs Python 3

- Python 3 is the standard for current Odoo development.
- Old examples on the internet may use Python 2 syntax.

A terminal window with a dark background and light green text. The window title bar shows three colored circles (red, yellow, green) and the text 'python3'. The terminal content shows a user named 'trainee' at a machine named 'odoo-lab' running two commands: 'python3 --version' which outputs 'Python 3.10.12', and 'which python3' which outputs '/usr/bin/python3'.

```
python3  
  
trainee@odoo-lab:~$ python3 --version  
Python 3.10.12  
trainee@odoo-lab:~$ which python3  
/usr/bin/python3
```

2 Section 2

Basic values and text

2.1 Numbers

- Python supports integers for whole numbers and floats for decimal values.
- Arithmetic operators include plus, minus, multiply, divide, and exponent.

```
python3

>>> customer_name = 'Agrolait'
>>> invoice_count = 3
>>> invoice_count = invoice_count + 1
>>> print(customer_name, invoice_count)
Agrolait 4
```

2.2 Variables

- A variable is a name bound to a value.
- Choose names that explain what the value represents.

```
python3

>>> customer_name = 'Agrolait'
>>> invoice_count = 3
>>> invoice_count = invoice_count + 1
>>> print(customer_name, invoice_count)
Agrolait 4
```

2.3 Strings

- A string is text surrounded by quotes.
- Single quotes and double quotes both work when used consistently.

```
python3
>>> customer_name = 'Agrolait'
>>> invoice_count = 3
>>> invoice_count = invoice_count + 1
>>> print(customer_name, invoice_count)
Agrolait 4
```

2.4 String concatenation

- Concatenation joins strings together with the plus operator.
- Both sides must be strings or Python raises a type error.

```
python3

>>> customer_name = 'Agrolait'
>>> invoice_count = 3
>>> invoice_count = invoice_count + 1
>>> print(customer_name, invoice_count)
Agrolait 4
```

2.5 f-strings

- An f-string starts with f before the quote.
- Expressions inside braces are evaluated and inserted into the text.

```
python3

>>> customer_name = 'Agrolait'
>>> invoice_count = 3
>>> invoice_count = invoice_count + 1
>>> print(customer_name, invoice_count)
Agrolait 4
```

2.6 String indexes

- Indexes select one character from a string.

First character

```
code = 'INV001'  
print(code[0])
```

- Python indexes start at zero.

Last character

```
code = 'INV001'  
print(code[-1])
```

2.7 String slicing

- A slice extracts part of a string.

Get prefix

```
number = 'INV/2026/001'  
print(number[:3])
```

- The start index is included and the stop index is not included.

Get year

```
number = 'INV/2026/001'  
print(number[4:8])
```

2.8 String immutability

- Strings are immutable, so their characters cannot be changed in place.

Create a changed copy

```
name = 'odoo'  
upper_name = name.upper()  
print(upper_name)
```

- String methods return new strings instead of editing the original.

Replace text

```
code = 'S0001'  
code = code.replace('S0', 'INV')  
print(code)
```

2.9 Type conversion

- Type conversion creates a value of another type.

Convert text to integer

```
quantity = int('5')  
print(quantity + 2)
```

- Common conversion functions include str, int, float, and bool.

Convert number to text

```
order_id = 42  
print('Order ' + str(order_id))
```

2.10 Booleans

- A boolean is either True or False.

Store active state

```
is_active = True  
print(is_active)
```

- Booleans are used for decisions and status flags.

Compare amount

```
amount = 250  
print(amount > 100)
```

3 Section 3

Collections of data



3.1 Lists

- A list stores multiple values in order.
- Lists can grow, shrink, and be changed after creation.

```
python3

>>> partners = ['Agrolait', 'Azure Interior']
>>> partners.append('Deco Addict')
>>> print(partners[0], len(partners))
Agrolait 3
```

3.2 List slicing

- List slicing returns a new list containing part of another list.
- The start index is included and the stop index is not included.

```
python3  
  
>>> partners = ['Agrolait', 'Azure Interior']  
>>> partners.append('Deco Addict')  
>>> print(partners[0], len(partners))  
Agrolait 3
```

3.3 append - Add to a list

- append adds one item to the end of a list.
- It changes the existing list in place.

```
python3  
  
>>> partners = ['Agrolait', 'Azure Interior']  
>>> partners.append('Deco Addict')  
>>> print(partners[0], len(partners))  
Agrolait 3
```

3.4 pop - Remove from a list

- pop removes an item from a list and returns it.
- Without an index, pop removes the last item.

```
python3

>>> partners = ['Agrolait', 'Azure Interior']
>>> partners.append('Deco Addict')
>>> print(partners[0], len(partners))
Agrolait 3
```

3.5 Dictionaries

- A dictionary stores key and value pairs.
- Keys are used to look up related values.

```
python3

>>> person = {'name': 'Ada', 'role': 'Admin'}
>>> print(person['name'])
Ada
>>> print(person.get('email', 'missing'))
missing
```

3.6 dict.get

- get reads a dictionary value without raising an error for a missing key.
- You can provide a default value as the second argument.

```
python3

>>> person = {'name': 'Ada', 'role': 'Admin'}
>>> print(person['name'])
Ada
>>> print(person.get('email', 'missing'))
missing
```

3.7 dict.keys

- keys shows the keys currently stored in a dictionary.
- It is useful when inspecting unfamiliar data.

```
python3

>>> person = {'name': 'Ada', 'role': 'Admin'}
>>> print(person['name'])
Ada
>>> print(person.get('email', 'missing'))
missing
```

3.8 Tuples

- A tuple stores ordered values like a list.

Create a tuple

```
status = ('draft', 'Draft')  
print(status)
```

- Tuples are immutable, so their contents cannot be changed in place.

Unpack a tuple

```
code, label = ('posted', 'Posted')  
print(label)
```

3.9 Sets

- A set stores unique values with no guaranteed order.

Remove duplicates

```
tags = {'vip', 'vip', 'retail'}  
print(tags)
```

- Adding a duplicate value keeps only one copy.

Find shared items

```
a = {'sale', 'stock'}  
b = {'stock', 'account'}  
print(a & b)
```

4 Section 4

Control flow

4.1 if, elif, and else

- if runs code only when a condition is true.
- elif checks another condition when the previous one was false.

A terminal window with a dark background and light green text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text 'python3'. The code entered is: >>> score = 85, >>> if score >= 50:, ... print('Pass'). The output shown is 'Pass' on a new line.

```
python3

>>> score = 85
>>> if score >= 50:
...     print('Pass')
Pass
```

4.2 Truthy and falsey values

- Python treats some non-boolean values as true or false in conditions.
- Empty strings, empty lists, empty dictionaries, zero, and None are falsey.



```
python3

>>> score = 85
>>> if score >= 50:
...     print('Pass')
Pass
```

4.3 for loops

- A for loop repeats code for each item in a sequence.
- The loop variable receives one item at a time.

A screenshot of a Python terminal window titled 'python3'. The window has a dark background with light green text. It shows a for loop that iterates over the list [1, 2, 3] and prints each item. The output shows the numbers 1, 2, and 3 on separate lines.

```
>>> for n in [1, 2, 3]:  
...     print(n)  
1  
2  
3
```

4.4 range

- range creates a sequence of numbers for looping.
- The stop value is not included.



```
python3

>>> for n in [1, 2, 3]:
...     print(n)
1
2
3
```

4.5 enumerate

- enumerate gives both the index and the item during a loop.
- It avoids manually updating a counter.

A terminal window with a dark background and light green text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text 'python3'. The terminal contains the following text:

```
>>> for n in [1, 2, 3]:  
...     print(n)  
1  
2  
3
```

4.6 while loops

- A while loop repeats while a condition stays true.
- The loop must change something or it may run forever.

A screenshot of a Python terminal window titled 'python3'. The window has a dark background with light green text. It shows a for loop that iterates over the list [1, 2, 3] and prints each element. The output shows the numbers 1, 2, and 3 on separate lines.

```
>>> for n in [1, 2, 3]:  
...     print(n)  
1  
2  
3
```

4.7 break

- break exits the nearest loop immediately.

Stop at match

```
for name in ['A', 'B', 'C']:
    if name == 'B':
        break
# ...
```

- Use it when the work is finished before the loop naturally ends.

Find first large amount

```
for amount in [20, 150, 10]:
    if amount > 100:
        print(amount)
# ...
```

4.8 continue

- continue skips the rest of the current loop step.

Skip empty names

```
for name in ['Agrolait', '', 'Deco']:
    if not name:
        continue
# ...
```

- The loop then moves to the next item.

Skip draft records

```
for state in ['draft', 'posted']:
    if state == 'draft':
        continue
# ...
```

5 Section 5

Functions and reusable code

5.1 Functions

- A function is a named block of reusable code.
- Use def to create a function.

A terminal window with a dark background and light green text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text 'python3'. The terminal contains the following text:

```
>>> def greet(name):  
...     return f'Hello, {name}'  
>>> print(greet('Ada'))  
Hello, Ada
```

5.2 Parameters

- Parameters are names that receive values passed into a function.
- They let one function work with different input values.

```
python3

>>> def greet(name):
...     return f'Hello, {name}'
>>> print(greet('Ada'))
Hello, Ada
```

5.3 Default parameters

- A default parameter value is used when no argument is provided.
- Defaults make common calls shorter.

A terminal window with a dark background and light green text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text 'python3'. The terminal contains the following text:

```
>>> def greet(name):  
...     return f'Hello, {name}'  
>>> print(greet('Ada'))  
Hello, Ada
```

5.4 return

- return sends a result back to the caller.
- Code after return in the same function is not executed.



```
python3

>>> def greet(name):
...     return f'Hello, {name}'
>>> print(greet('Ada'))
Hello, Ada
```

5.5 *args

- *args collects extra positional arguments into a tuple.
- Use it when a function may receive a flexible number of values.

```
python3

>>> def greet(name):
...     return f'Hello, {name}'
>>> print(greet('Ada'))
Hello, Ada
```

5.6 **kwargs

- **kwargs collects extra named arguments into a dictionary.
- It is common in framework code that passes flexible options.

```
python3  
  
>>> def greet(name):  
...     return f'Hello, {name}'  
>>> print(greet('Ada'))  
Hello, Ada
```

5.7 Scope

- Scope describes where a name can be used.

Local variable

```
def build_name():  
    name = 'Agrolait'  
    return name  
# ...
```

- Names created inside a function are local to that function.

Pass outside value

```
prefix = 'INV'  
def make_number(number):  
    return f'{prefix}{number}'  
# ...
```

6 Section 6

Objects, errors, modules, and Odoo habits

6.1 Classes and `__init__`

- A class defines a kind of object with data and behavior.
- `__init__` runs when a new object is created.



```
python3

>>> class User:
...     def __init__(self, name):
...         self.name = name
>>> print(User('Ada').name)
Ada
```

6.2 Methods

- A method is a function defined inside a class.
- Methods use self to read or change object data.

```
python3

>>> class User:
...     def __init__(self, name):
...         self.name = name
>>> print(User('Ada').name)
Ada
```

6.3 Inheritance

- Inheritance lets one class reuse behavior from another class.
- The child class can add or replace methods.

```
python3

>>> class User:
...     def __init__(self, name):
...         self.name = name
>>> print(User('Ada').name)
Ada
```

6.4 super

- super calls behavior from a parent class.
- Use it when extending a method instead of replacing everything.

```
python3

>>> class User:
...     def __init__(self, name):
...         self.name = name
>>> print(User('Ada').name)
Ada
```

6.5 try and except

- try runs code that may fail.
- except handles a specific error so the program can respond.



```
python3

>>> try:
...     int('abc')
... except ValueError:
...     print('Bad number')
Bad number
```

6.6 Modules and import

- A module is a Python file that can be imported by other code.

Import a standard module

```
import datetime
print(datetime.date.today())
```

- import loads names from modules so you can reuse existing functionality.

Import one name

```
from pathlib import Path
print(Path.cwd())
```

6.7 Virtual environments

- A virtual environment keeps Python packages separate for one project.
- It helps avoid conflicts between system packages and project packages.

```
user@odoo-lab:~/project

trainee@odoo-lab:~/project$ python3 -m venv .venv
trainee@odoo-lab:~/project$ source .venv/bin/activate
(.venv) trainee@odoo-lab:~/project$ pip install requests
Successfully installed requests-2.32.3
```

6.8 pip install

- pip installs Python packages into the active environment.
- Use `python -m pip` so pip belongs to the interpreter you are using.

```
user@odoo-lab:~/project

trainee@odoo-lab:~/project$ python3 -m venv .venv
trainee@odoo-lab:~/project$ source .venv/bin/activate
(.venv) trainee@odoo-lab:~/project$ pip install requests
Successfully installed requests-2.32.3
```

6.9 Reading Python errors

- Python tracebacks show where an error happened and what type it was.
- Read from the bottom for the final error message.

A screenshot of a Python terminal window titled 'python3'. The window has a dark background with light green text. It shows a try-except block where an attempt is made to convert the string 'abc' to an integer, which fails, resulting in a ValueError. The except block prints the message 'Bad number'.

```
>>> try:
...     int('abc')
... except ValueError:
...     print('Bad number')
Bad number
```

6.10 Debugging basics

- Debugging means finding why actual behavior differs from expected behavior.

Print value and type

```
amount = '100'  
print(amount, type(amount))
```

- Print small facts such as type, value, and branch taken.

Use breakpoint

```
amount = 100  
breakpoint()  
print(amount * 2)
```

6.11 Python habits for Odoo

- Write small readable methods that match Odoo model responsibilities.

Name values clearly

```
partner_vals = {'name': 'Agrolait', 'customer_rank': 1}
```

- Prefer clear names for recordsets, dictionaries, and computed values.

Use a readable helper

```
def is_large_invoice(amount):  
    return amount >= 1000
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory
02 · Python

